ARTIFICIAL INTELLIGENCE CSA1701 LAB OBSERVATON
Mohammed Fareeth S A (192511288)

EXPERIMENT 8:

Aim

To implement and solve the classical AI Vacuum World problem using the Depth-First Search (DFS) graph search algorithm to find a valid sequence of actions from an initial state to the goal state.

Algorithm

1. **Define State Space:** Represent each state as a tuple containing the agent's location and the dirt status of all rooms (e.g., (location, status_a, status_b)).
2. **Initialize Frontier and Explored Set:**
    - Create a stack (frontier) containing the starting state and the path (list of actions) taken to reach it.
    - Initialize an empty set for explored states to prevent infinite loops and redundant paths.
3. **Stack Processing (DFS Loop):**
    - While the frontier is not empty, pop the most recently added state and its corresponding path from the stack.
    - If the current state satisfies the goal condition (all rooms are clean), return the path.
4. **State Expansion:**
    - If the state has not been explored, mark it as explored.
    - Generate all possible legal successor states based on valid actions (Left, Right, Suck) and push them onto the stack along with the updated path.
5. **Terminate:** Repeat until the goal state is found and output the resulting action sequence.

CODE:

```python
class VacuumDFS:
    def __init__(self, initial_location, initial_a, initial_b):
        self.initial_state = (initial_location, initial_a, initial_b)

    def is_goal(self, state):
        _, status_a, status_b = state
        return not status_a and not status_b

    def get_successors(self, state):
        location, status_a, status_b = state
        successors = []

        if location == 'A':
            successors.append(('Suck', ('A', False, status_b)))
            successors.append(('Right', ('B', status_a, status_b)))
        else:
            successors.append(('Suck', ('B', status_a, False)))
            successors.append(('Left', ('A', status_a, status_b)))

        return successors

    def search(self):
        frontier = [(self.initial_state, [])]
        explored = set()

        while frontier:
            current_state, path = frontier.pop()

            if self.is_goal(current_state):
                return path

            if current_state not in explored:
                explored.add(current_state)
                for action, next_state in self.get_successors(current_state):
                    if next_state not in explored:
                        frontier.append((next_state, path + [action]))

        return None

problem = VacuumDFS(initial_location='A', initial_a=True, initial_b=True)
path = problem.search()

print(f"Initial State: ('A', True, True)")
print("Goal State Reached!\n")
print("Action Sequence:")
for i, action in enumerate(path, 1):
    print(f"{i}. {action}")

print(f"\nTotal Cost (Steps): {len(path)}")
```

```
============= RESTART: C:/Users/Admin/OneDrive/Documents/aiexp8.py ==
Initial State: ('A', True, True)
Goal State Reached!

Action Sequence:
1. Right
2. Suck
3. Left
4. Suck

Total Cost (Steps): 4
```

Result

The DFS algorithm successfully explored the state space starting from a state where the agent is at room *A* and both rooms are dirty ('A', True, True). By diving deep into branches using a LIFO (Last-In, First-Out) stack structure, the algorithm identified a valid sequence of actions required to reach the goal state.

Conclusion

The program successfully demonstrated the application of Depth-First Search to the Vacuum World problem. While DFS successfully finds a path to the goal, the resulting path may not always be the shortest or most optimal (unlike BFS), because it explores branches deeply before backtracking.